Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Florianópolis
2027

Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Universidade Federal de Santa Catarina

Orientador: Antônio Augusto Fröhlich

Florianópolis
2027

LISTA DE ILUSTRAÇÕES

Figura 1 – Exemplo Básico de Níveis de Privilégio. 5
Figura 2 – Hipervisor Tipo 1. 6
Figura 3 – Hipervisor Tipo 2. 7

SUMÁRIO

1	INTRODUÇÃO	4
2	FUNDAMENTAÇÃO TEÓRICA	5
2.1	Níveis de Privilégio	5
2.2	Virtualização	5
2.3	Hipervisor	6
2.3.1	Tipo 1	6
2.3.2	Tipo 2	7
2.4	Interceptação e Emulação - Trap and Emulate	7
3	METODOLOGIA	8
	REFERÊNCIAS	10

1 INTRODUÇÃO

Sistemas embarcados tornaram-se a espinha dorsal da infraestrutura tecnológica moderna, estando presentes em uma ampla variedade de aplicações, desde dispositivos médicos e sistemas de automação industrial até plataformas automotivas e missões espaciais. Diferentemente da computação de propósito geral, esses sistemas frequentemente operam sob requisitos estritos de tempo real, nos quais a corretude do comportamento não depende apenas do resultado lógico produzido, mas também do instante exato em que esse resultado é disponibilizado.

Tradicionalmente, a garantia desse determinismo esteve associada ao uso de plataformas dedicadas, nas quais Sistemas Operacionais de Tempo Real (RTOSs) são empregados para assegurar previsibilidade temporal, isolamento entre tarefas e cumprimento de restrições rígidas de escalonamento. Entretanto, a evolução do software contemporâneo deslocou grande parte dos ecossistemas de desenvolvimento — bibliotecas, frameworks, ferramentas e serviços — para ambientes baseados em Sistemas Operacionais de Propósito Geral (GPOSs), especialmente Linux. Embora esses sistemas ofereçam grande riqueza funcional e amplo suporte da comunidade, sua adoção em contextos críticos demanda esforços adicionais de adaptação, validação e integração.

Como resultado, cresce a necessidade de consolidar, em uma mesma plataforma computacional, cargas de trabalho com requisitos distintos e frequentemente conflitantes: de um lado, aplicações críticas, sensíveis a prazos temporais e dependentes de comportamento determinístico; de outro, aplicações complexas, orientadas a funcionalidades avançadas e dependentes de ambientes generalistas.

Nesse cenário, técnicas de virtualização surgem como uma alternativa promissora, pois permitem a coexistência de múltiplos ambientes de execução sobre o mesmo hardware. Dessa forma, torna-se possível executar aplicações críticas e não críticas de maneira controlada, reduzindo custos de integração e aumentando o aproveitamento da plataforma.

Paralelamente, a arquitetura aberta RISC-V vem ganhando relevância no domínio de sistemas embarcados devido à sua flexibilidade, modularidade e independência tecnológica. Seu conjunto de instruções extensível, aliado à ausência de barreiras proprietárias, favorece tanto a pesquisa acadêmica quanto o desenvolvimento industrial, especialmente em soluções customizadas e de baixo custo.

Por isso, este trabalho propõe o projeto e a implementação, em linguagem C++, de uma base de virtualização para sistemas embarcados baseados em RISC-V, com foco em previsibilidade temporal, isolamento entre cargas de trabalho e baixo custo de execução. Busca-se, assim, contribuir para a viabilização de plataformas embarcadas capazes de integrar aplicações heterogêneas sem comprometer requisitos críticos de desempenho e confiabilidade.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 NÍVEIS DE PRIVILÉGIO

Os níveis de privilégio surgem com a evolução dos Sistemas Operacionais (SOs) e com a necessidade de particionar e proteger os recursos de um computador. Esses níveis são organizados em um modelo de camadas, no qual a camada mais alta possui acesso mais restrito, enquanto a camada mais baixa detém controle total sobre o hardware. Além disso, como os níveis superiores não possuem acesso direto a determinados recursos, os níveis inferiores podem lhes fornecer serviços, abstrações e mecanismos de acesso, sempre de forma controlada. Embora a implementação desses níveis varie entre diferentes arquiteturas, não é desejável que todas as aplicações executem no nível mais privilegiado, pois isso pode comprometer o isolamento entre processos e gerar competição descontrolada pelos recursos do sistema. Por isso, a maioria dos processadores e SOs adotam, no mínimo, uma divisão binária com o modo usuário e o modo supervisor, ilustrados na Figura 1.

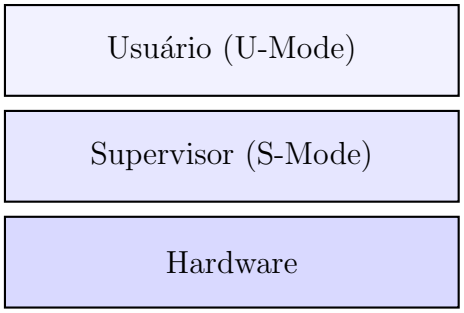


Figura 1 – Exemplo Básico de Níveis de Privilégio.

O modo usuário é destinado à execução da maioria das aplicações. Nesse nível, os programas operam com acesso restrito aos recursos do sistema, sem executar instruções privilegiadas nem acessar diretamente componentes críticos da máquina.

Já o modo supervisor é destinado à execução do próprio SO, dispondo de acesso total aos recursos do sistema. Nesse nível, o sistema operacional gerencia memória, dispositivos e processos, atuando como intermediário entre os programas em modo usuário e o hardware.

2.2 VIRTUALIZAÇÃO

Para computação, virtualização é a inserção de uma camada adicional entre um mediador de hardware e o hardware físico. Quando um componente é virtualizado, esse recurso específico deixa de ser utilizado de forma direta e passa a ser apresentado por meio de uma abstração lógica controlada por essa camada intermediária. Isso permite,

por exemplo, que múltiplas aplicações utilizem simultaneamente o mesmo recurso, pois a camada de virtualização fica responsável por arbitrar o acesso e coordenar seu compartilhamento evitando conflitos.

2.3 HIPERVISOR

Enquanto SOs tradicionalmente atuam como supervisores (Figura 1), um hipervisor introduz uma camada adicional de abstração. Ele opera um nível de privilégio abaixo dos SOs convidados, bloqueando o acesso direto deles ao hardware e virtualizando os recursos físicos para orquestrar múltiplos SOs de forma isolada.

Para que essa estrutura funcione, o hipervisor utiliza mecanismos de interceptação. Como o SO convidado opera acreditando deter o privilégio máximo sobre a máquina, ele tenta executar instruções privilegiadas diretamente. No entanto, como ele não possui esse controle essas ações geram uma exceções que são capturadas pelo hipervisor. Este, por sua vez, traduz ou emula a operação, garantindo que o acesso aos recursos físicos ocorra de forma segura e isolada.

Fundamentalmente, existem dois tipos de hipervisors: 1 e 2.

2.3.1 Tipo 1

Hipervisores de tipo 1, também conhecidos como bare-metal, são aqueles que executam diretamente sobre o hardware físico, sem a necessidade de um sistema operacional hospedeiro intermediário. Eles atuam como SO e como camada de virtualização apresentando maior desempenho e menor latência em comparação aos hipervisores do Tipo 2. Além disso, proporcionam maior controle sobre os recursos do sistema e melhor isolamento entre as máquinas virtuais, tornando-os especialmente adequados para ambientes críticos.

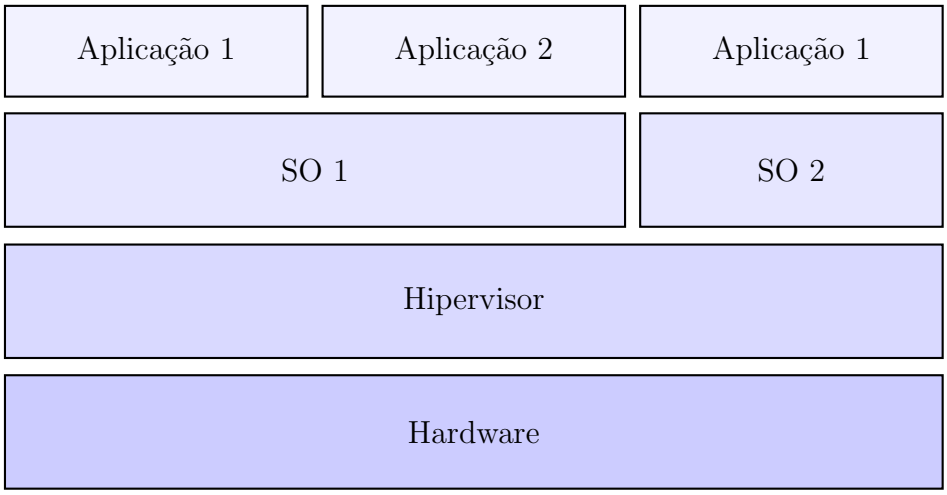


Figura 2 – Hipervisor Tipo 1.

2.3.2 Tipo 2

Diferentemente da solução nativa, um hipervisor de Tipo 2 é executado como um processo dentro de um SO hospedeiro. Apesar de garantir o isolamento e a virtualização para diversos SOs convidados, sua arquitetura impõe mais uma camada intermediária de software que restringe a eficiência no acesso aos componentes físicos, gerando latência adicional e menor desempenho em operações críticas.

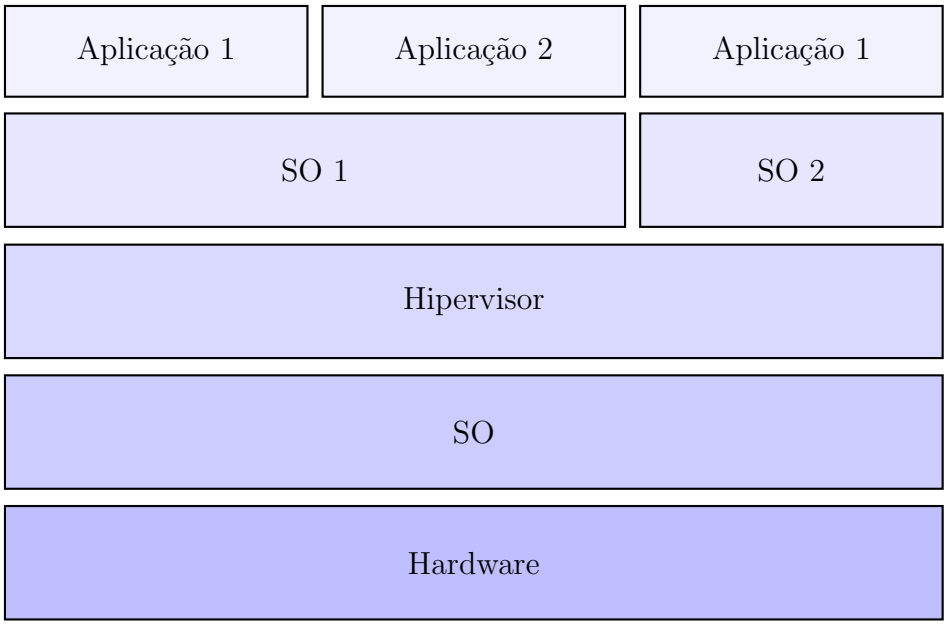


Figura 3 – Hipervisor Tipo 2.

2.4 INTERCEPTAÇÃO E EMULAÇÃO - TRAP AND EMULATE

3 METODOLOGIA

GLOSSÁRIO

Hardware Parte física e tangível de um computador. 5–7

Mediador de Hardware Um componente de software que encapsula, gerencia e abstrai dispositivos físicos. 5

RISC-V Conjunto de instruções de máquina livre para ser usado para qualquer finalidade, permitindo a qualquer pessoa ou empresa projetar e vender chips RISC-V sem precisar pagar royalties. 4

Sistema Embarcado Um computador pequeno feito para executar uma tarefa específica dentro de outro equipamento. 4

REFERÊNCIAS